Taylor–Hood θ-schemes and adaptive SUPG for the time-dependent Navier–Stokes equations: implementation, observations and results

galerkit, branch `time-navier-stokes`

November 2026

Abstract

This report documents two pieces of code written on the galerkit branch `time-navier-stokes` (package `galerkit.navier_stokes.twod.theta_th`). Part I is a fresh Taylor–Hood P_2/P_1 implementation of the coupled θ-schemes of John [1], Sect. 7.3.1, without any stabilisation, together with the examples of that section (Examples 7.44, 7.57 in two dimensions, 7.58 and 7.62, based on Examples D.5, D.7, D.8, D.9 of Appendix D). Part II extends the adaptive SUPG method of de Frutos, García-Archilla, John and Novo [2] from evolutionary convection–diffusion–reaction equations to the Navier–Stokes equations, with the inf-sup stable Taylor–Hood pair (no pressure stabilisation), and reports first simulations for the smooth solution of Example D.7. All rates and comparisons below are *observed numerically*; nothing in this report is a proof. The verdict for Part II is that the implementation is verified and the adaptive loop works, but on the smooth test problem no adaptive variant is more efficient than uniform refinement, and the direct extension of the FGJN14 error estimator localises poorly.

Contents

1 Common discretisation **2**

I θ-schemes after John (2016), Chapter 7 **2**

2 Implementation **2**

3 Example D.5: steady flow around a cylinder (validation) **3**

4 Example D.7: smooth solution, convergence **3**

5 Example 7.44: pressure error with Crank–Nicolson **4**

6 Example 7.57 in two dimensions: Picard iteration vs Newton's method **4**

7 Example 7.58: flow around a cylinder with time-dependent inflow (Example D.9) **5**

8 Example 7.62: IMEX schemes (Example D.8) **6**

II Adaptive SUPG for Navier–Stokes (extension of FGJN14) **6**

9 Discretisation **6**

10 Adaptive algorithm **7**

11 Results for the smooth solution (Example D.7) **8**
11.1 Uniform meshes and effectivity 8
11.2 Adaptive runs 9
11.3 Diagnosis of the `supg` indicator 9

12 Observations **9**

13 A solver artefact: inaccurate PARDISO solves **11**

14 Summary **11**

1 Common discretisation

We consider, in a bounded polygon $\Omega \subset \mathbb{R}^2$,

$$\partial_t \boldsymbol{u} - \nu \Delta \boldsymbol{u} + (\boldsymbol{u} \cdot \nabla)\boldsymbol{u} + \nabla p = \boldsymbol{f}, \qquad \nabla \cdot \boldsymbol{u} = 0, \qquad \boldsymbol{u}(0) = \boldsymbol{u}_0, \tag{1}$$

with Dirichlet data on part of the boundary and, for the flows around a cylinder, the do-nothing condition $\nu \partial_{\boldsymbol{n}} \boldsymbol{u} - p\boldsymbol{n} = \boldsymbol{0}$ at the outlet.

Space. Continuous P_2 velocity and continuous P_1 pressure on triangles (Taylor–Hood). Curved boundaries (the cylinder) are represented isoparametrically: every cell is mapped from the reference triangle by the quadratic map defined by its six P_2 nodes, with edge midpoints on curved boundary parts placed on the exact boundary. Integrals use a collapsed Gauss rule exact for polynomials of degree 7 on the reference triangle. The viscous term is in gradient form, the convective term in convective form $n(\boldsymbol{w}, \boldsymbol{u}, \boldsymbol{v}) = ((\boldsymbol{w} \cdot \nabla)\boldsymbol{u}, \boldsymbol{v})$ unless the skew-symmetric form $\frac{1}{2}[((\boldsymbol{w} \cdot \nabla)\boldsymbol{u}, \boldsymbol{v}) - ((\boldsymbol{w} \cdot \nabla)\boldsymbol{v}, \boldsymbol{u})]$ is stated.

Meshes. Unit square: 2×2 squares, two triangles each with alternating diagonals, uniformly refined; level ℓ has mesh width $h = \sqrt{2}\, 2^{-(\ell+1)}$. Cylinder channel $(0, 2.2) \times (0, 0.41)$ minus the disc of radius 0.05 at $(0.2, 0.2)$: a transfinite O-grid around the cylinder and a Cartesian block downstream, split into triangles and refined uniformly; levels 1, 2, 3 have 5 082, 19 668, 77 352 unknowns. The front and back points of the cylinder are mesh vertices on every level.

Linear algebra. Sparse direct solver: PARDISO (via `pypardiso`) when installed, SciPy's SuperLU otherwise. PARDISO perturbs small pivots of the saddle-point matrices; every solve therefore checks $\|b - Ax\|_2 \le 10^{-12}\|b\|_2$, refines iteratively up to three times with the cached factorisation and falls back to SuperLU (Sect. 13).

Part I
θ-schemes after John (2016), Chapter 7

2 Implementation

Module `theta_th.solver`, class `ThetaTH`. One substep from t_k to $t_{k+1} = t_k + \Delta t_{k+1}$ inside the macro step of length Δt_{n+1} is the variational problem [1, (7.84)]:

$$\begin{aligned}
&(\boldsymbol{u}_{k+1}, \boldsymbol{v}) + \theta_1 \Delta t_{n+1}\big[\nu(\nabla \boldsymbol{u}_{k+1}, \nabla \boldsymbol{v}) + n(\boldsymbol{u}_{k+1}, \boldsymbol{u}_{k+1}, \boldsymbol{v})\big] - \Delta t_{k+1}(\nabla \cdot \boldsymbol{v}, p_{k+1}) \\
&\qquad = (\boldsymbol{u}_k, \boldsymbol{v}) - \theta_2 \Delta t_{n+1}\big[\nu(\nabla \boldsymbol{u}_k, \nabla \boldsymbol{v}) + n(\boldsymbol{u}_k, \boldsymbol{u}_k, \boldsymbol{v})\big] + \theta_3 \Delta t_{n+1}(\boldsymbol{f}_k, \boldsymbol{v}) + \theta_4 \Delta t_{n+1}(\boldsymbol{f}_{k+1}, \boldsymbol{v}), \\
&\qquad 0 = \Delta t_{k+1}(\nabla \cdot \boldsymbol{u}_{k+1}, q),
\end{aligned}$$

with the divergence constraint scaled by Δt_{k+1} as in all simulations of [1] (Remark 7.50). The book prints "2ν" in the θ_2 term of (7.84); (7.80)–(7.83) and (7.85) have ν, which is used here. The parameters $\theta_1, \dots, \theta_4$ are those of Tables 7.1 and 7.2 (module `theta_th.schemes`): backward Euler, Crank–Nicolson, the general one-step θ-scheme, and the fractional-step schemes FS0 and FS1 with $\theta = 1 - \sqrt{2}/2$, $\tilde{\theta} = 1 - 2\theta$, $\tau = \tilde{\theta}/(1-\theta)$, $\eta = 1 - \tau$.

Nonlinear solvers: Picard iteration (7.85) and Newton's method (7.86), initial iterate $(\boldsymbol{u}_k, p_k)$, stopped when the Euclidean norm of the residual of (7.84) on the non-Dirichlet rows is below 10^{-10}; IMEX variants (Remark 7.61) with the constant extrapolation $\boldsymbol{u}_{\text{prev}} = \boldsymbol{u}_k$ or the linear extrapolation (7.89), the explicit θ_2 term using the convection field of the previous step as in (7.91).

Verification (unit tests, `tests/test_ns2d_theta_th.py`). Tables 7.1/7.2 reproduced; stability functions $R(z)$ with $R(z) - e^z = O(z^{p+1})$, $R(-\infty) = -1$ (CN), 0 (BE), $-\eta/\tau \approx -0.71$ (FS0); exactness for a steady P_2/P_1 solution; time orders 1 (BE) and 2 (CN, FS0, FS1) for a solution in the discrete space; IMEX orders 2 (linear) and 1 (constant extrapolation); quadratic convergence of Newton's method.

3 Example D.5: steady flow around a cylinder (validation)

Re = 20, inflow $1.2\,y(0.41 - y)/0.41^2$, do-nothing outflow, drag and lift by the volume formulas (D.16), (D.17).

level	unknowns	$c_d - c_{d,\text{ref}}$	$c_l - c_{l,\text{ref}}$	$\Delta p - \Delta p_{\text{ref}}$	Newton steps
0	1 353	$+3.5 \cdot 10^{-2}$	$+8.0 \cdot 10^{-2}$	$+4.0 \cdot 10^{-3}$	5
1	5 082	$-9.7 \cdot 10^{-4}$	$-5.2 \cdot 10^{-4}$	$+6.1 \cdot 10^{-4}$	5
2	19 668	$-2.4 \cdot 10^{-4}$	$-1.6 \cdot 10^{-4}$	$+2.1 \cdot 10^{-4}$	5
3	77 352	$-1.7 \cdot 10^{-5}$	$-1.2 \cdot 10^{-5}$	$+5.9 \cdot 10^{-5}$	5

References (D.19), (D.21), (D.22): $c_d = 5.57953523384$, $c_l = 0.010618948146$ (do-nothing), $\Delta p = 0.11752016697$.

4 Example D.7: smooth solution, convergence

$\Omega = (0,1)^2$, $\boldsymbol{u} = 2\pi \sin t\,(\sin^2(\pi x)\sin(\pi y)\cos(\pi y),\ -\sin(\pi x)\cos(\pi x)\sin^2(\pi y))$, $p = 20 \sin t\,(x^2 y - 1/6)$, homogeneous Dirichlet data, $\nu = 10^{-2}$, $T = 1$.

Space (FS0, $\Delta t = 0.005$). The column "p shifted" compares p_h with p at the time the last substep's pressure approximates, $t_{k+1} - (1 - \theta_1/(\theta_1 + \theta_2))\Delta t_{k+1}$.

level	$\|\boldsymbol{u} - \boldsymbol{u}_h\|_0$	rate	$\|\nabla(\boldsymbol{u} - \boldsymbol{u}_h)\|_0$	rate	$\|p - p_h\|_0$ at T	rate	p shifted	rate
0	4.30e-1		6.34		6.07e-1		6.07e-1	
1	1.31e-1	1.72	3.22	0.98	1.60e-1	1.93	1.59e-1	1.93
2	1.61e-2	3.02	0.822	1.97	2.81e-2	2.51	2.81e-2	2.51
3	1.98e-3	3.02	0.195	2.07	6.53e-3	2.11	6.39e-3	2.13
4	2.23e-4	3.15	0.0436	2.16	2.00e-3	1.71	1.52e-3	2.07

Observed: the optimal Taylor–Hood rates 3, 2, 2. The dip of the pressure rate at T on level 4 is the first-order temporal pressure error of FS0 at t_{n+1}; it disappears at the shifted time.

Time (level 4). Velocity error at T against the same scheme with $\Delta t = 1/640$ on the same mesh (temporal error only):

Δt	BE	rate	CN	rate	FS0	rate	FS1	rate
0.2	7.39e-2		4.07e-3		4.85e-4		1.87e-3	
0.1	3.65e-2	1.02	1.02e-3	2.00	1.20e-4	2.01	2.67e-4	2.81
0.05	1.80e-2	1.03	2.54e-4	2.00	3.00e-5	2.01	5.38e-5	2.31
0.025	8.68e-3	1.05	6.32e-5	2.00	7.47e-6	2.01	1.29e-5	2.06
0.0125	4.05e-3	1.10	1.56e-5	2.02	1.85e-6	2.02	3.10e-6	2.06

Observed: orders 1 (BE) and 2 (CN, FS0, FS1). Against the exact solution all second-order schemes level off at $2.23 \cdot 10^{-4}$, the spatial error of level 4. The pressure is of first order at t_{n+1} for every scheme (Remark 7.50); the Crank–Nicolson pressure compared at $t_n + \Delta t/2$ converges with order 2 down to the spatial floor $\approx 1.5 \cdot 10^{-3}$; the FS0 pressure at $t_{n+1} - \eta\theta\Delta t$ is at the spatial floor already for $\Delta t = 0.1$ (observed, no explanation proved); the FS1 pressure stays of first order there (plausibly because its last substep evaluates $\boldsymbol{f}$ only at the old time, $\theta_4 = 0$; not verified).

5 Example 7.44: pressure error with Crank–Nicolson

Book: Q_2/P_1^{disc}, $\Delta t = 10^{-4}$, $T = 5$. Here: Taylor–Hood P_2/P_1, Crank–Nicolson with $\Delta t = 10^{-3}$, $T = 5$, the discrete norm $\left(\Delta t \sum_n \|p(t) - p_h^n\|_0^2\right)^{1/2}$ with p evaluated at t_n or at $t_n - \Delta t/2$.

	$\nu = 1$				$\nu = 10^{-2}$			
level	at t_n	rate	at $t_n - \Delta t/2$	rate	at t_n	rate	at $t_n - \Delta t/2$	rate
0	1.65		1.65		1.25		1.25	
1	0.754	1.13	0.754	1.13	0.319	1.96	0.319	1.96
2	8.71e-2	3.11	8.71e-2	3.11	5.45e-2	2.55	5.44e-2	2.55
3	1.42e-2	2.62	1.39e-2	2.65	1.27e-2	2.10	1.24e-2	2.14
4	4.28e-3	1.73	3.01e-3	2.20	4.23e-3	1.59	2.94e-3	2.07

Observed (convective form): second order when the Crank–Nicolson pressure is compared at the midpoint, as in Fig. 7.2 of the book, and errors nearly independent of ν for $\nu \in \{1, 10^{-2}\}$. Comparing at t_n adds a temporal error $\approx \frac{\Delta t}{2}\|\partial_t p\|$ that dominates on level 4.

For $\nu = 10^{-4}$ the unstabilised scheme with the convective form blows up on level 1 ($\|\boldsymbol{u}_h\|$ grows to 15 at $t \approx 2.5$, Newton then fails); the convective form is not energy stable for the only weakly divergence-free Taylor–Hood velocity (plausible cause, not proved). With the skew-symmetric form the computation is stable, but the pressure error stays large: 1.25, 7.18, 4.07, 3.85 on levels 0–3 (level 4 was not completed within the 2 h limit of the runs). The book's observation that the pressure error does not depend on ν does not hold for this discretisation at $\nu = 10^{-4}$ on these meshes.

6 Example 7.57 in two dimensions: Picard iteration vs Newton's method

The book's Example 7.57 is three-dimensional (Example D.10). Here: Example D.7, $\nu = 10^{-3}$, Crank–Nicolson with $\Delta t = 0.01$ on $[0, 0.1]$, solution of the previous time as initial iterate, stopping tolerance 10^{-10}, direct solver.

level	unknowns	method	linear systems (10 steps)	solve time [s]	$\|\boldsymbol{u}(T)-\boldsymbol{u}_h\|_0$
2	659	Picard	29	0.27	2.406e-3
2	659	Newton	20	0.17	2.406e-3
3	2 467	Picard	28	0.81	6.369e-4
3	2 467	Newton	20	1.42	6.369e-4
4	9 539	Picard	27	4.15	1.262e-4
4	9 539	Newton	20	3.56	1.262e-4
5	37 507	Picard	26	15.97	1.714e-5
5	37 507	Newton	20	11.40	1.714e-5

As in Table 7.3 of the book, Newton's method needs fewer linear systems (two per step instead of about three). The timings were taken on a shared machine and include the residual checks of the solver; they are indicative only.

7 Example 7.58: flow around a cylinder with time-dependent inflow (Example D.9)

$\nu = 10^{-3}$, inflow $\sin(\pi t/8)\,6y(0.41-y)/0.41^2$ on $[0,8]$, $\boldsymbol{u}_0 = \boldsymbol{0}$, do-nothing outflow as in Example 7.58, level 2 (19 668 unknowns), Newton's method, tolerance 10^{-10}. The book uses Q_2/P_1^{disc} on levels 3–5 (37 216 to 588 160 unknowns).

scheme, Δt	$c_{d,\max}$	t	$c_{l,\max}$	t	$\Delta p(8)$	linear solves
BE, 0.02	2.950769	3.9320	0.001159	–	−0.126729	800
BE, 0.01	2.950513	3.9319	0.023069	7.3443	−0.124637	1 600
BE, 0.005	2.950404	3.9320	0.166381	6.3505	−0.107584	3 200
CN, 0.02	2.950700	3.9418	0.444633	5.7997	−0.098212	948
CN, 0.01	2.950758	3.9382	0.462118	5.7615	−0.103691	1 754
CN, 0.005	2.950771	3.9363	0.466132	5.7508	−0.105762	3 200
FS0, 0.02	2.950792	3.9362	0.464448	5.7539	−0.105187	2 413
FS0, 0.01	2.950784	3.9352	0.466521	5.7483	−0.106273	4 466
FS0, 0.005	2.950778	3.9347	0.466880	5.7466	−0.106606	7 257
Table D.3	2.950918381	3.93625	0.47787543	5.69250	−0.11161567	

Table D.3 was computed with the Dirichlet outflow (D.29) on fine meshes; the remaining gap of $c_{l,\max}$ is mostly spatial. Errors (discrete $l^2(0,8)$) to the FS0 run with $\Delta t = 0.005$ on the same mesh (temporal errors only):

scheme, Δt	c_d	c_l	Δp	linear solves
BE, 0.005	1.14e-1	4.05e-1	6.24e-2	3 200
CN, 0.02	1.91e-2	2.95e-1	2.27e-2	948
CN, 0.01	8.13e-3	8.61e-2	9.24e-3	1 754
CN, 0.005	3.39e-3	2.41e-2	3.66e-3	3 200
FS0, 0.02	3.48e-3	4.23e-2	4.07e-3	2 413
FS0, 0.01	1.10e-3	9.69e-3	1.22e-3	4 466

Observed, as in the book: backward Euler is by far the least accurate and misses the vortex shedding for $\Delta t \geq 0.005$ (Fig. 1, cf. Fig. 7.5); Crank–Nicolson converges with order about two; FS0 is 5–9 times more accurate than Crank–Nicolson at equal Δt and still 2–2.5 times more accurate at comparable cost (the reference is FS0 itself, which favours FS0; against a Crank–Nicolson reference the ordering is the same). The book reports a slightly better accuracy per CPU time for Crank–Nicolson (Remark 7.59, John and Rang 2010); that is not reproduced here, which depends on the solver.

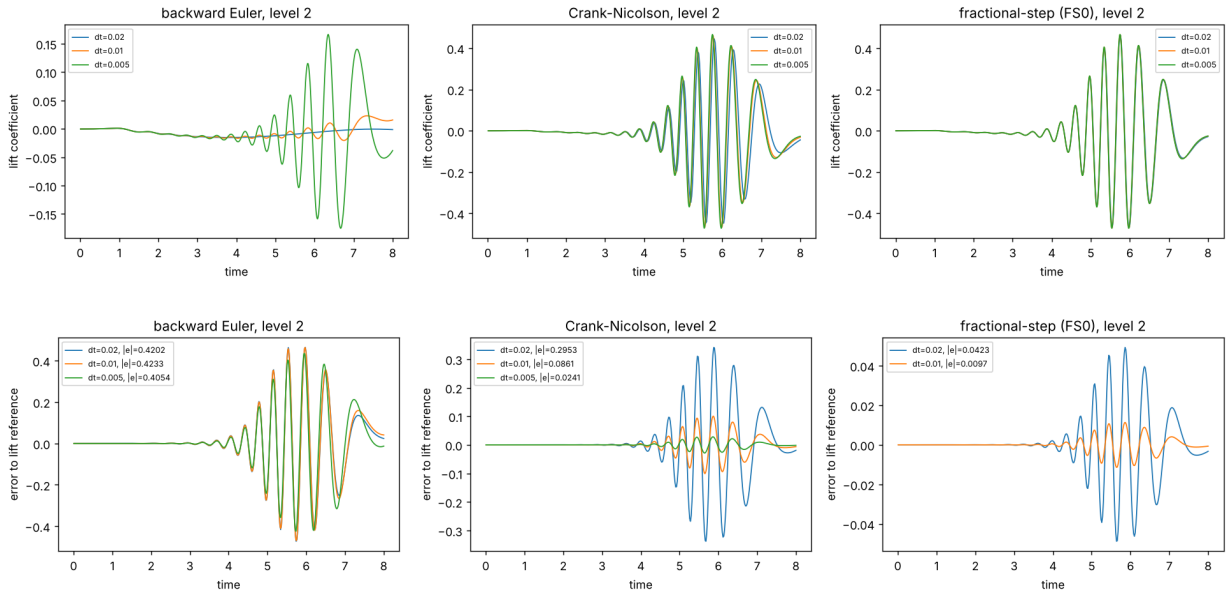


Figure 1: Example 7.58 / D.9, level 2: lift coefficient (top) and its error to the FS0 run with $\Delta t = 0.005$ (bottom).

8 Example 7.62: IMEX schemes (Example D.8)

$\mathrm{Re} = 100$, steady inflow $6y(0.41 - y)/0.41^2$, spin-up with Crank–Nicolson ($\Delta t = 0.01$) to $t = 15$, then two time units with Crank–Nicolson, $\Delta t = 0.005$, level 2.

method	$c_{d,\max}$	$c_{l,\max}$	St	$\Delta p(t_0 + T/2)$	linear solves
fully implicit (Newton)	3.2222	0.9841	0.2984	2.4858	800
IMEX, linear extrapolation	3.2341	1.0052	0.3008	2.4930	400
IMEX, constant extrapolation	3.4010	1.5833	0.2857	2.5892	400
Table D.2	[3.22, 3.24]	[0.98, 1.02]	[0.295, 0.305]	[2.46, 2.50]	

Observed, as in Fig. 7.10: the constant extrapolation is quite inaccurate (and its amplitude still grows), the linear extrapolation gives almost the same results as the fully implicit scheme with half the linear solves; both lie inside all four reference intervals on level 2. On level 1 (5 082 unknowns) the flow is not resolved (St ≈ 0.27).

Part II
Adaptive SUPG for Navier–Stokes (extension of FGJN14)

9 Discretisation

Module `theta_th.supg`, class `SUPGTH`. With the transport field $\boldsymbol{w} = \boldsymbol{u}_h$ (fully nonlinear) and the discrete time derivative $\boldsymbol{u}_h'$,

$$(\boldsymbol{u}_h', \boldsymbol{v}) + \nu(\nabla \boldsymbol{u}_h, \nabla \boldsymbol{v}) + ((\boldsymbol{w}\cdot\nabla)\boldsymbol{u}_h, \boldsymbol{v}) - (p_h, \nabla\cdot\boldsymbol{v}) \\ + \sum_K \delta_K \big(\boldsymbol{u}_h' - \nu\Delta\boldsymbol{u}_h + (\boldsymbol{w}\cdot\nabla)\boldsymbol{u}_h + \nabla p_h - \boldsymbol{f}, (\boldsymbol{w}\cdot\nabla)\boldsymbol{v}\big)_K = (\boldsymbol{f}, \boldsymbol{v}), \qquad (\nabla\cdot\boldsymbol{u}_h, q) = 0.$$

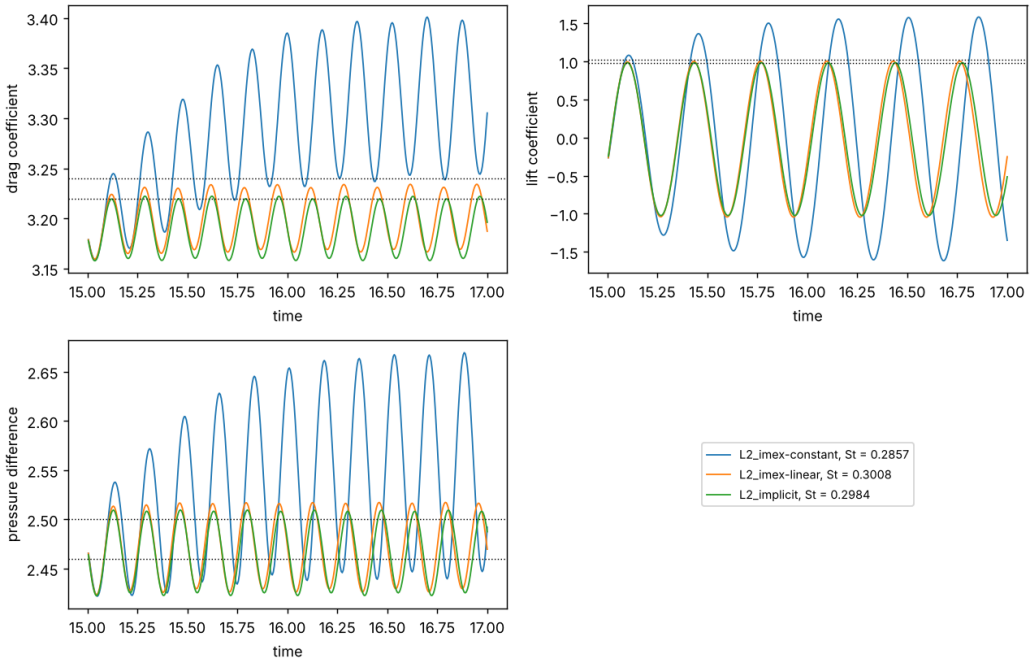


Figure 2: Example 7.62 / D.8, level 2: drag, lift and pressure difference; dotted: reference intervals of Table D.2.

The pressure gradient is part of the consistent residual; there is *no* pressure stabilisation (no PSPG, no grad-div): Taylor–Hood is inf-sup stable. Time: the trapezoidal rule with the discrete derivative as in [2], Sect. 3.3, $\boldsymbol{u}_{n+1} = \boldsymbol{u}_n + \frac{\Delta t}{2}(\boldsymbol{u}'_n + \boldsymbol{u}'_{n+1})$, the equation collocated at t_{n+1}; with $\delta_K = 0$ this is algebraically Crank–Nicolson (reproduced to all digits). Stabilisation parameter (a choice of this implementation; [2] use $\delta_0 h_K/\|\boldsymbol{b}\|_{\infty,K}$):

$$\delta_K = \delta_0 \min\left\{h_K/\|\boldsymbol{w}_0\|_{\infty,K},\, h_K^2/\nu\right\}, \qquad \delta_0 = 0.25,$$

computed from the initial iterate $\boldsymbol{w}_0 = \boldsymbol{u}_n + \Delta t\, \boldsymbol{u}'_n$ of the step and kept fixed in the nonlinear iteration. Newton's method with the full Jacobian (Galerkin convection and both SUPG derivatives $\delta_K((\delta\boldsymbol{u}\cdot\nabla)\boldsymbol{u}, (\boldsymbol{u}\cdot\nabla)\boldsymbol{v})$ and $\delta_K(R(\boldsymbol{u}), (\delta\boldsymbol{u}\cdot\nabla)\boldsymbol{v})$), an Armijo line search and a Picard fallback; stopping criterion $\|r\|_2 \le 10^{-8} + 10^{-10}\|b\|_2$.

Verification (`tests/test_ns2d_adaptive_supg_th.py`). A steady P_2/P_1 solution is reproduced to round-off for $\delta_0 \in \{0, 0.25, 5\}$ (consistency); Newton's method converges quadratically; the transfer between meshes is exact under refinement; all residuals vanish for a discrete solution.

10 Adaptive algorithm

Module `theta_th.adaptive`, class `AdaptiveSUPGNS`, following the steps of [2], Sect. 3.3, as implemented for convection–diffusion in `galerkit.convection_diffusion.adaptive_supg` (the PDF of [2] was not at hand; details of its p. 227 not covered there may differ). After every accepted time step:

1. TR step; step size by the TR-AB2 controller of Gresho, Griffiths and Silvester: AB2 predictor, $d = (\boldsymbol{u} - \boldsymbol{u}^p)/(3(1 + \Delta t_{\text{old}}/\Delta t))$ in the L^2 norm of the velocity, $\Delta t_{\text{new}} = 0.85\Delta t(\text{tol}_t/\|d\|)^{1/3}$, rejection if $\|d\| > \text{tol}_t/0.7^3$, averaging every 10 steps, $\Delta t_0 = 10^{-10}$;

2. local indicators η_K (an `ErrorIndicator`);

3. marking (a `Marking`): FGJN14 thresholds ($\eta_K > \mathrm{TOL}_1$ refine, $\eta_K < \mathrm{TOL}_2 = \mathrm{TOL}_1/20$ coarsen) or Dörfler marking (below);

4. newest-vertex bisection and its exact inverse;

5. transfer of $\boldsymbol{u}_h$ by interpolation (exact under refinement).

Additions needed for Navier–Stokes (each documented in the code with the failure it prevents): (i) after a mesh change $\boldsymbol{u}_h'$ and p_h are recomputed from the discrete equation (an interpolated $\boldsymbol{u}_h'$ violates it by $O(1)$; 325 rejections in 4 accepted steps were observed); (ii) the step may grow by at most a factor 10 (from $\Delta t_0 = 10^{-10}$ the predictor is exact and the unlimited formula proposed $\Delta t \approx 10^{88}$); (iii) the nonlinear iteration starts from the predictor, and a failed step is repeated with $\Delta t/2$; (iv) δ_K is computed from the predictor ($\boldsymbol{u}(0) = \mathbf{0}$ in Example D.7 gave $\delta_K = \delta_0 h^2/\nu \approx 80$ otherwise).

Error indicators, designed to be extended. All residual indicators share

$$R_K = \boldsymbol{f} - \boldsymbol{u}_h' + \nu\Delta\boldsymbol{u}_h - (\boldsymbol{u}_h\cdot\nabla)\boldsymbol{u}_h - \nabla p_h, \qquad R_E = -\nu[\![\partial_{\boldsymbol{n}}\boldsymbol{u}_h]\!], \qquad R_{\mathrm{div}} = \nabla\cdot\boldsymbol{u}_h,$$

and differ in their weights (a `Weights` subclass):

$$\eta_K^2 = w_K\|R_K\|_K^2 + \sum_{E\subset\partial K} w_E\|R_E\|_E^2 + w_{\mathrm{div},K}\|\nabla\cdot\boldsymbol{u}_h\|_K^2.$$

name	weights	origin
`supg`	$w_K = \min\{h_K^2/\nu, 24\delta_K\} + 24\delta_K$, $w_E = \min\{24/\|\boldsymbol{u}_h\|_{\infty,E}, h_E/\nu\}$, $w_{\mathrm{div}} = \nu$	[2, (28)], $\varepsilon\to\nu$, $\boldsymbol{b}\to\boldsymbol{u}_h$, $c\to 0$
`h1`	h_K^2, h_E, 1	energy-norm weights
`l2`	h_K^4, h_E^3, h_K^2	L^2 weights
`exact`	local SUPG-norm error of a known solution	for testing

The divergence term is an addition of this extension (Taylor–Hood is only weakly divergence free). A new indicator is a new `Weights` or `ErrorIndicator` subclass registered in `INDICATORS`; the marking strategy is exchangeable as well, and a separate `global_indicator` may supply the global estimate.

11 Results for the smooth solution (Example D.7)

$T = 1$, $\delta_0 = 0.25$, $\mathrm{tol}_t = 10^{-3}$, initial mesh 4×4 squares. Error measures at T: $\|\boldsymbol{u} - \boldsymbol{u}_h\|_0$ and the SUPG norm $(\nu|\boldsymbol{u} - \boldsymbol{u}_h|_1^2 + \sum_K \delta_K\|(\boldsymbol{u}_h\cdot\nabla)(\boldsymbol{u} - \boldsymbol{u}_h)\|_K^2)^{1/2}$.

11.1 Uniform meshes and effectivity

ν, method	$\|\boldsymbol{u} - \boldsymbol{u}_h\|_0$, levels 1→4	rates	SUPG norm, levels 1→4
10^{-2}, SUPG	1.22e-1 → 7.73e-4	2.47, 2.50, 2.34	6.56e-1 → 2.11e-2 (rates 1.66, 1.62, 1.68)
10^{-2}, Galerkin	1.31e-1 → 2.24e-4	3.02, 3.02, 3.15	–
10^{-4}, SUPG	1.60e-1 → 7.75e-3	0.92, 1.39, 2.06	6.94e-1 → 4.75e-2 (rates 1.26, 1.31, 1.30)
10^{-4}, Galerkin	3.05e-1 → 3.98e-3	1.41, 2.08, 2.77	–

(Unknowns on levels 1–4: 188, 660, 2 468, 9 540. The "SUPG norm" of a Galerkin run lacks the δ term and is not listed.) On these meshes SUPG is less accurate than Galerkin in L^2 for this resolved smooth solution, and its SUPG-norm rates (1.3–1.7) are below the $h^{k+1/2} = h^{2.5}$ of the theory for convection-dominated problems; whether this is pre-asymptotic or an effect of the choice of δ_K is open.

Effectivity $\eta/\|e\|_{\mathrm{SUPG}}$ on uniform meshes (levels 1–4, after 20 steps of $\Delta t = 0.01$):

ν	`supg`	`h1`	`l2`
10^{-2}	11.6, 13.4, 15.6, 17.7	5.4, 7.0, 8.0, 8.7	1.9, 1.2, 0.7, 0.4
10^{-4}	10.0, 8.2, 8.5, 9.1	6.7, 12.9, 24.5, 41.2	2.4, 2.3, 2.2, 1.8

The extended FGJN14 estimator is robust in ν (as reported in [2] for convection–diffusion), the `h1` weights are not.

11.2 Adaptive runs

ν	marking	cells	unknowns	steps (rej.)	$\|\boldsymbol{u}-\boldsymbol{u}_h\|_0$	SUPG norm	max Newton
10^{-2}	`supg`, TOL_1 0.04	482	2 318	43 (0)	5.92e-3	7.72e-2	3
10^{-2}	`supg`, TOL_1 0.02	857	4 093	41 (0)	3.08e-3	4.71e-2	3
10^{-2}	`supg`, TOL_1 0.01	1 569	7 337	38 (0)	1.38e-3	2.62e-2	3
10^{-4}	`supg`, TOL_1 0.04	2 028	9 560	74 (3)	2.20e-2	8.27e-2	21
10^{-4}	`supg`, TOL_1 0.02	2 193	10 270	53 (2)	1.63e-2	9.60e-2	18
10^{-4}	`supg`, TOL_1 0.01	4 027	18 738	47 (3)	1.17e-2	5.86e-2	30
10^{-4}	`exact`, TOL_1 5e-3	1 108	5 160	55 (0)	2.08e-2	8.34e-2	5
10^{-4}	`exact`, TOL_1 2.5e-3	2 303	10 610	47 (0)	1.19e-2	5.00e-2	8
10^{-4}	`exact`, TOL_1 1.25e-3	5 455	24 839	47 (0)	8.76e-3	2.94e-2	6
10^{-4}	Dörfler, `h1`/`supg`, 0.5	4 442	20 413	68 (0)	9.89e-3	4.10e-2	7
10^{-4}	Dörfler, `h1`/`supg`, 0.3	7 583	34 670	67 (0)	8.90e-3	2.89e-2	7
10^{-4}	Dörfler, `h1`/`supg`, 0.2	8 021	36 661	67 (0)	8.84e-3	2.70e-2	7
10^{-4}	uniform level 4	2 048	9 540	100 ($\Delta t = 0.01$)	7.75e-3	4.75e-2	3

Dörfler marking (`DoerflerMarking`, $\theta = 0.3$): if the global `supg` estimate exceeds the target, refine the smallest set of cells carrying 30 % of $\sum m_K^2$, m_K the `h1` indicator; in every step the cells carrying the smallest 5 % are coarsening candidates. The three runs with `supg` marking were repeated with the corrected linear solver (Sect. 13) with unchanged results.

11.3 Diagnosis of the `supg` indicator

On the same SUPG states (uniform levels 3, 4, $t = 0.5$) η_K was compared cell by cell with the exact local SUPG-norm error e_K (`diagnose_indicator.py`). For $\nu = 10^{-4}$, level 4:

weights	effectivity	Spearman(η_K, e_K)	top-10 % overlap	slow cells' share of η^2 (of e^2: 0.39)
`supg`	6.6	0.41	0.62	0.85
`supg`, δ_K capped at $\delta_0 h_K/U_\infty$	2.5	0.47	0.67	0.65
cell weight h_K^2, `supg` edge	1.7	0.66	0.74	0.89
edge + divergence terms only	1.7	0.70	0.74	0.89
`h1`	36.8	0.79	0.79	0.71

("slow": $\|\boldsymbol{u}_h\|_{\infty,K} < 0.2$.) For $\nu = 10^{-2}$, level 4: `supg` effectivity 14.7, Spearman 0.51, top-10 % overlap 0.21; `h1` 7.95, 0.97, 0.76. At $\nu = 10^{-4}$ the cell part of η^2 is 37–41 times $\sum e_K^2$, the edge part 0.2–2.7 times, the divergence part at most 0.14 times.

12 Observations

1. **The cell weight of the extended FGJN14 estimator is unbounded where $\boldsymbol{u}_h \to 0$.** With $\delta_K = \delta_0 h_K/\|\boldsymbol{u}_h\|_{\infty,K}$ the weight $\min\{h_K^2/\nu, 24\delta_K\} + 24\delta_K \approx 48\delta_K$ grows like $h_K/|\boldsymbol{u}_h|$; δ_K/h_K reaches 65 near walls and vortex centres. In [2] the convection $\boldsymbol{b}$ is given and typically bounded away from zero; in Navier–Stokes it is not. At $\nu = 10^{-4}$, level 4, cells with $\|\boldsymbol{u}_h\|_{\infty,K} < 0.05$ carry 44 % of η^2 and 5.5 % of e^2. The estimator is robust globally but ranks the cells poorly; this explains its poor adaptive meshes (non-monotone errors, up to 30 Newton iterations).

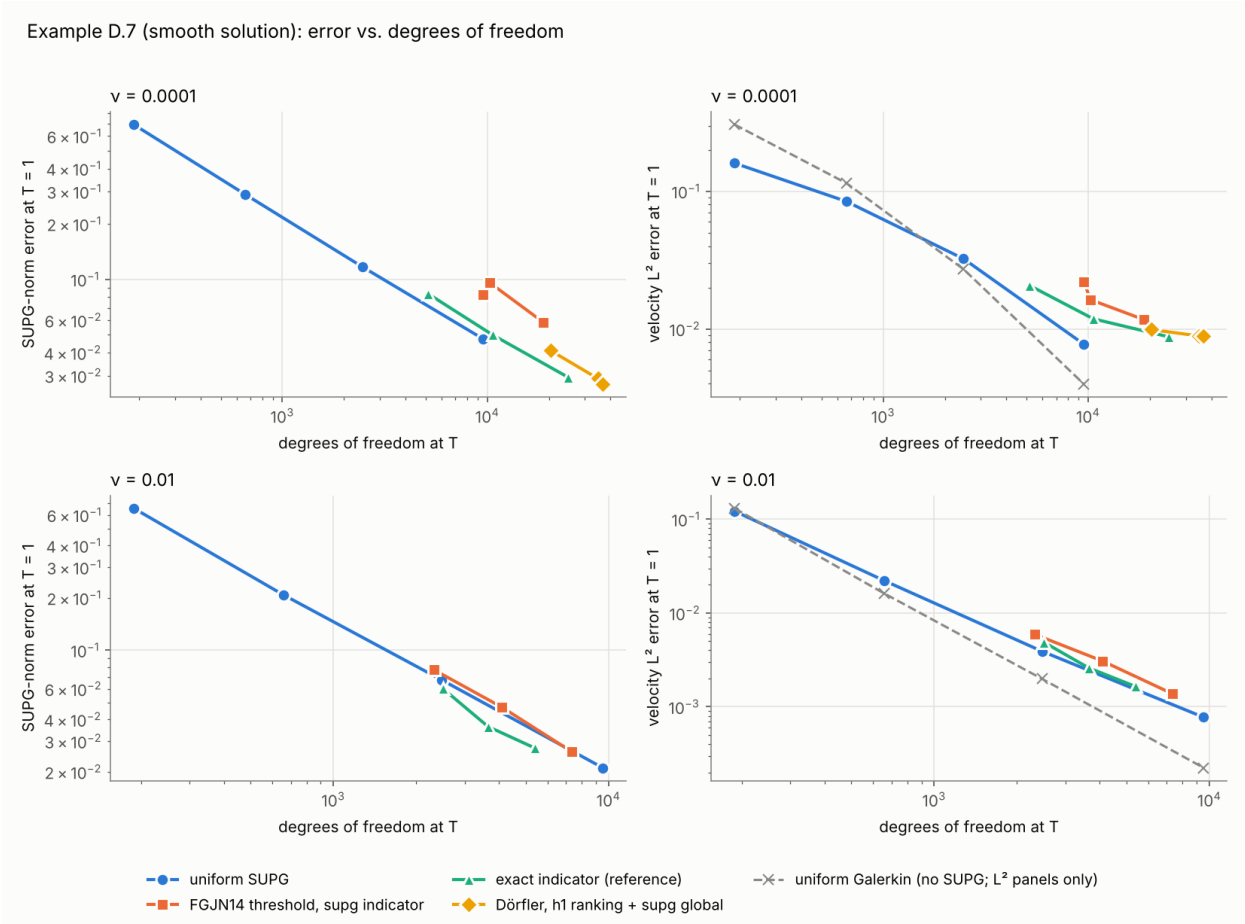


Figure 3: Example D.7: error at $T = 1$ against the number of unknowns ($\nu = 10^{-4}$ top, 10^{-2} bottom).

2. **Marking and global estimate must be separated.** The `h1` indicator ranks the cells best but its magnitude is not robust in ν; with Dörfler marking on `h1` and the `supg` global estimate the adaptive runs at $\nu = 10^{-4}$ behave as intended (monotone errors, no rejections, at most 7 Newton iterations). The control run with Dörfler marking on `supg` collapsed: its 5 % "smallest" cells are the fast-flow cells, where the error is; the mesh was coarsened there (157 → 45 cells), the error grew from 0.15 to 0.28, the estimate exploded to $\sim 10^5$ and the step size to $\sim 10^{-10}$ near $t = 0.414$.

3. **On the smooth solution adaptivity does not pay off.** At $\nu = 10^{-4}$ the exact indicator reaches the uniform level-4 SUPG-norm error at similar cost and beats it at 2.6 times the unknowns; `h1`/Dörfler needs about 1.4 times the unknowns of the exact indicator. This is expected for a smooth solution and says nothing about problems with layers.

4. **Probable time-error floor.** The adaptive L^2 errors at $\nu = 10^{-4}$ level off near $9 \cdot 10^{-3}$ while the unknowns double (Fig. 3); the adaptive runs take 47–68 steps ($\mathrm{tol}_t = 10^{-3}$) against 100 fixed steps for the uniform runs. A rerun with $\mathrm{tol}_t = 10^{-4}$ would settle this; not done.

5. **SUPG vs Galerkin.** For this resolved smooth solution SUPG is less accurate than Galerkin in L^2 on uniform meshes, and its SUPG-norm rates are below theory; open (pre-asymptotic behaviour or the choice of δ_K).

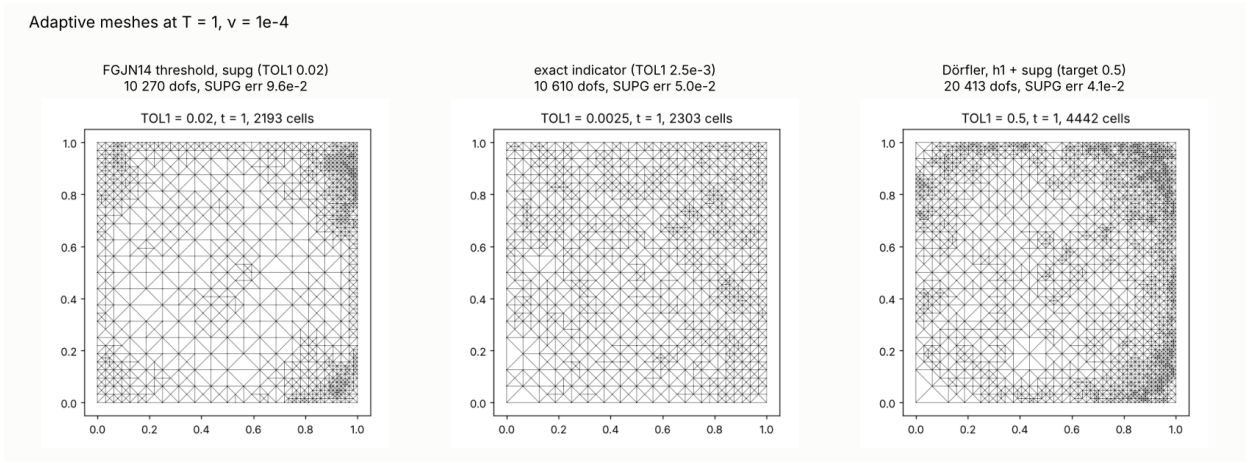


Figure 4: Adaptive meshes at $T = 1$, $\nu = 10^{-4}$.

13 A solver artefact: inaccurate PARDISO solves

With PARDISO, the adaptive run with the exact indicator and $\mathrm{TOL}_1 = 1.25 \cdot 10^{-3}$, executed as the third run in one process, ended with a SUPG-norm error of 0.20 instead of 0.029; the error jumped tenfold in the last steps. With SuperLU, the same sequence gives 0.029 (identical to a standalone run), and the first two runs agree to all printed digits. On the mass/divergence saddle-point system solved after each mesh change, PARDISO's relative residual was $8.5 \cdot 10^{-10}$ against $2 \cdot 10^{-14}$ for SuperLU. The residual-checked solve (iterative refinement, SuperLU fallback) restores the SuperLU result exactly; in that run 54 refinements and one fallback occurred in 589 solves. Which mechanism inside PARDISO (pivot perturbation or reused state) causes the inaccuracy was not established. With `pypardiso` installed, six existing tests of older galerkit modules also fail (precision), independently of this work.

14 Summary

Part I reproduces the qualitative findings of [1], Sect. 7.3.1, with Taylor–Hood P_2/P_1: optimal spatial rates for the smooth solution, temporal orders 1/2/2/2 for BE/CN/FS0/FS1, the pressure time levels of Remark 7.50, the inaccuracy of backward Euler and the higher accuracy of FS0 for the flow around a cylinder, Newton needing fewer linear systems than Picard, and the IMEX results of Example 7.62. Deviations: the element, coarser meshes, and the failure of the unstabilised scheme at $\nu = 10^{-4}$ in Example 7.44.

Part II gives a verified SUPG–Taylor–Hood discretisation with the trapezoidal rule and an adaptive algorithm with exchangeable indicators and marking strategies. The adaptive loop works with the exact indicator and with `h1` ranking + `supg` global estimate; the direct extension of the FGJN14 estimator localises poorly because its cell weight is unbounded where the velocity vanishes. Next steps: rerun with $\mathrm{tol}_t = 10^{-4}$; a test problem with layers, where adaptivity can pay off; an analysis of a cell weight that stays bounded for $\boldsymbol{u}_h \to 0$.

Reproduction. Code and scripts: galerkit, branch `time-navier-stokes`, `examples/navier_stokes/2d/theta_john2016` (`d5_steady.py`, `d7_convergence.py`, `picard_newton.py`, `d9_cylinder.py`, `d8_imex.py`) and `examples/navier_stokes/2d/adaptive_supg_ns` (`adaptive_d7.py`, `diagnose_indicator.py`, `plot_summary.py`); each script has `--smoke` and documents its options.

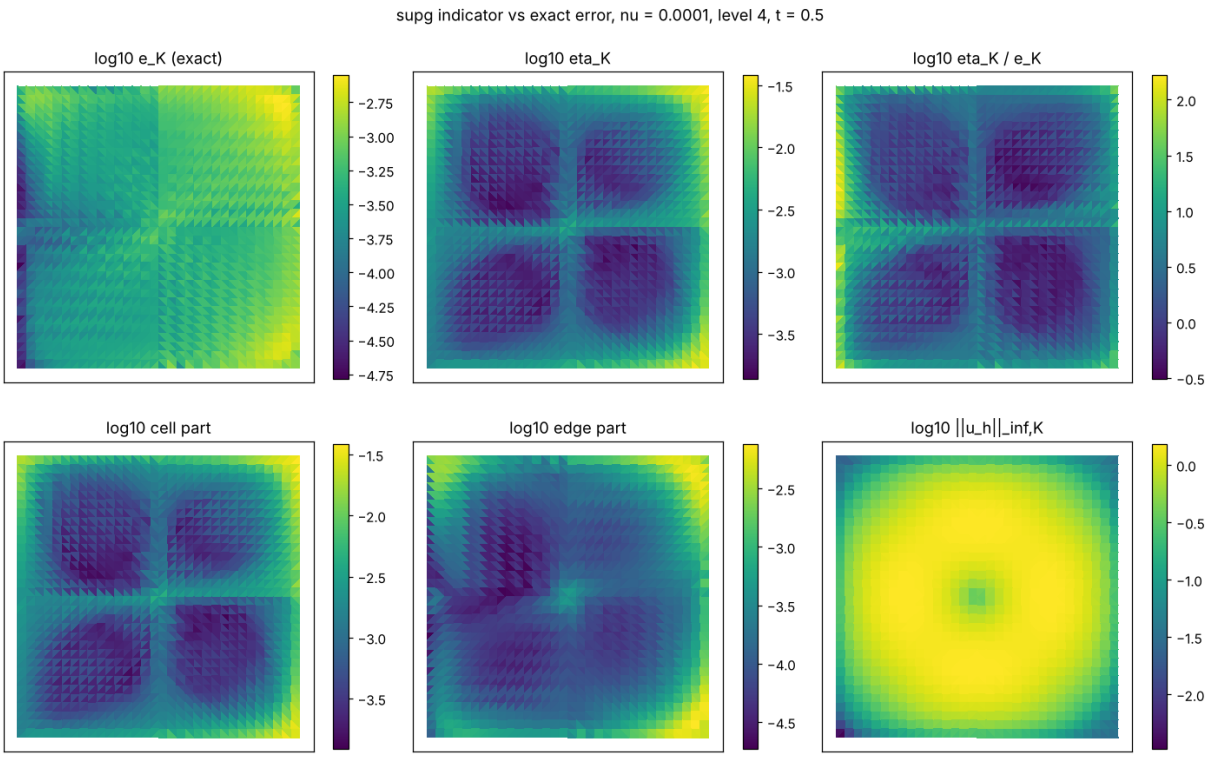


Figure 5: $\nu = 10^{-4}$, level 4, $t = 0.5$: exact local error, `supg` indicator, their ratio, cell and edge parts, local velocity (all $\log_{10}$).

References

[1] V. John, *Finite Element Methods for Incompressible Flow Problems*, Springer Series in Computational Mathematics 51, Springer, 2016.

[2] J. de Frutos, B. García-Archilla, V. John, J. Novo, An adaptive SUPG method for evolutionary convection–diffusion equations, *Comput. Methods Appl. Mech. Engrg.* 273 (2014) 219–237.